



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Standard data types-List	
Topic Code	-	
Unit/Module No. & Name	2	Standard Data Types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-6
3.0	Nature Or Type	7-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14-15



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. This paper explains the linked list, which is a basic data structure in computer science.
2. It is written for undergraduate students who are learning how programs store and organize data in memory.
3. The language is clear and familiar, so readers with basic programming knowledge can understand the ideas without advanced mathematics.
4. By the end, readers should understand what a node is, how nodes are connected by links, and how common operations like inserting, deleting, and searching work.
5. Readers should also understand why a linked list can be a better choice than an array in some cases and why it can be slower in other cases.
6. A linked list can be thought of as a chain of small records, where each record knows the location of the next record in memory.
7. Because the chain uses references, the items do not need to be stored next to each other in memory.
8. In practice, readers should be able to follow a pointer as it moves from one node to another and explain what happens in memory when a node is added or removed.
9. They should also be able to choose the right type of linked list, such as singly linked or doubly linked, based on the problem's needs.

1.0 INTRODUCTION

1.1 Why Data Structures Matter

In computer science, a data structure is a planned way to store data so that a program can work efficiently. A program often needs to add new data, delete old data, or find data quickly. If data is stored without planning, the program can waste time and memory. Data structures help balance speed, memory use, and simplicity. Common examples include arrays, stacks, queues, trees, and graphs, and each one is useful for different tasks.

For undergraduate students, linked lists are important because they clearly connect theory with how memory works in real systems. Learning linked lists is not just about storing values. It also helps students understand memory addresses, pointers, and dynamic memory allocation. This understanding supports later topics such as recursion, hashing, memory management, and performance analysis.

1.2 Where Linked Lists Appear

Linked lists are used in many parts of computing, even if users do not see them directly. Operating systems may use linked lists to manage free memory, running tasks, or open files. Programming languages may use them to manage objects, support garbage collection, or store temporary data. Applications may use linked lists for playlists, undo actions, or active connections. Linked lists are rarely used alone, but they often act as a basic building block in larger systems.

1.2.1 The Idea Of Dynamic Growth

One reason linked lists are taught early is that they show how data structures can grow dynamically. In an array, the size is usually fixed in advance, or resizing takes extra effort. In a linked list, a new node can be created whenever needed and linked to the list, making it longer. This flexibility is useful, although it also adds some cost due to memory allocation and pointer usage.

1.2.2 The Cost Of Pointer Chasing

Linked lists also help explain an important performance concept called locality. In arrays, elements are stored close together in memory, so the processor can access them quickly. In linked lists, nodes may be stored in different memory locations, so the processor may take more time to reach each one. This is why linked lists can be slower than arrays when simply scanning through data, even though their structure is easy to understand.

2.0 MEANING AND DEFINITION

2.1 Meaning In Plain Words

In plain terms, a linked list is like a treasure hunt where each clue tells you where to find the next clue. Each step gives you two things: the value you care about and a direction to the next step. You begin at the first clue, called the head, and you follow links until you reach the end. The end is marked by a special reference, often a null value, meaning there is no next node.

This image is simple, but it captures the central meaning. A list is not mainly defined by where items sit in memory. It is defined by relationships. Each node knows one neighbor, or sometimes two, and these local connections create a whole sequence. In computer science, this idea supports many other structures, including trees and graphs, where links build large shapes from small parts.

2.2 Formal Definition

A linked list is an ordered collection of nodes in which each node contains a data element and at least one link that references another node. The list is accessed through a reference to the first node, and the order of elements is defined by repeatedly following the links. A linked list is stored using dynamically allocated nodes, meaning nodes may be created and removed during program execution.

2.2.1 Node

A node is the basic unit of the list. In many languages, it is represented as a record or object with fields. One field stores the value, such as an integer, string, or a more complex object. Another field stores the link, often called next, that holds the address of the next node. In a doubly linked list, there is also a previous link.

A useful academic point is that the node's shape is a design choice. The data field can store the value itself, or it can store a pointer to a value stored elsewhere. The link field can store a direct address, an index, or another kind of reference, depending on the language and memory model.

2.2.2 Link

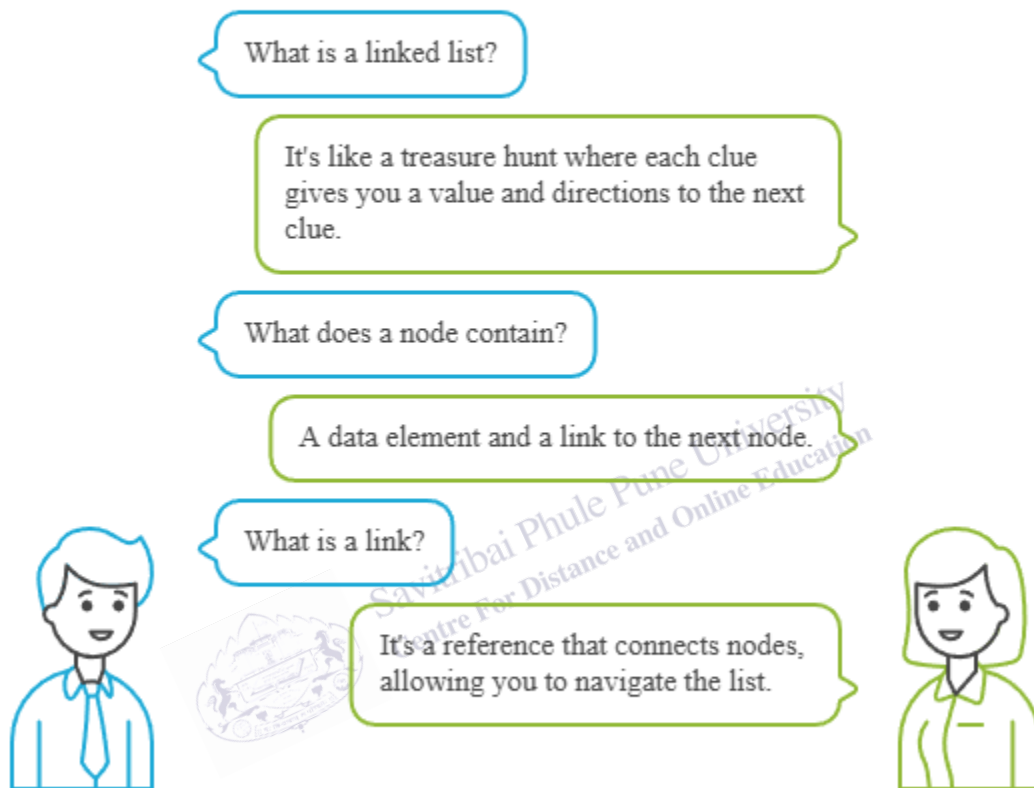
A link is the reference that connects nodes. In a low level language like C, it is commonly a pointer. In a managed language like Java, it is a reference that the runtime manages. In either case, the link gives the program a way to reach the next node. The link is not the same as the value; it is metadata that supports navigation.

Understanding links requires understanding two ideas. First, a link can be valid or null. A null link marks the end of a linear list. Second, links can be changed. By changing one or two link

fields, you can insert or remove nodes without moving the rest of the nodes in memory. This ability is the key advantage of linked lists.

Fig. 1

Linked Lists Explained



The image explains linked lists using simple dialogue, defining a linked list, a node, and a link. It uses conversational bubbles and characters to show how values connect through references, helping beginners understand navigation and structure without heavy technical detail.

3.0 NATURE OR TYPE

3.1 Core Nature

The nature of a linked list is relational and dynamic. Relational means that each element is tied to the next through a reference, so the structure exists because of connections, not because of position. Dynamic means the structure can change length during execution, as nodes are allocated and freed. These qualities make linked lists suitable when the number of elements is unknown or when frequent insertions and deletions are expected.

A linked list also has a clear access pattern. You can reach the head quickly because you store a direct reference to it. However, reaching the k th element usually requires walking from the head through k links. This pattern is called sequential access. It contrasts with random access in arrays, where you can compute an index address directly.

3.2 Singly Linked List

A singly linked list is the simplest common form. Each node stores data and one link to the next node. The last node's next link is null. Because each node points forward only, traversal is naturally forward. This form uses less memory per node than more complex forms, because it stores only one link.

Inserting a node after a given node is straightforward. You set the new node's next to the old next, and then set the current node's next to the new node. Deleting a node after a given node is also straightforward, as long as you have a reference to the node before the one you remove. The key limitation is that you cannot easily move backward without starting over from the head.

3.3 Doubly Linked List

A doubly linked list stores two links in each node: next and previous. This allows traversal in both directions. The head node's previous is null, and the tail node's next is null, in the standard linear form. The extra link costs extra memory, but it provides practical benefits.

With two links, deletion becomes easier when you already have a reference to the node you want to delete. You can reconnect the previous node to the next node and update both links. This matters in applications like browser histories or editors, where you want to move backward and forward and remove items efficiently.

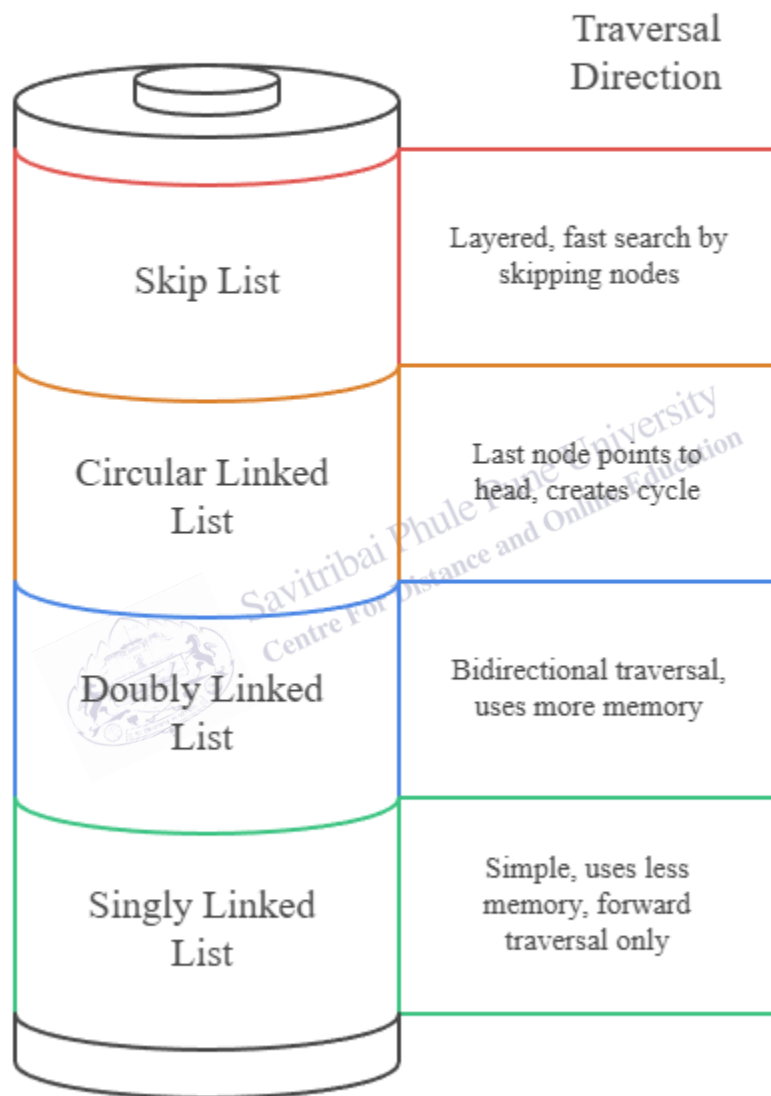
3.4 Circular Linked List

In a circular linked list, the last node does not point to null. Instead, it points back to a chosen node, often the head. This creates a cycle. Circular lists can be singly or doubly linked. They are useful when the data naturally repeats, such as in round robin scheduling.

A circular list changes how you detect the end. Because there is no null to stop you, you often stop when you return to the start node. This requires careful loop conditions in code. The benefit is that you can move from the tail to the head in one step and maintain a continuous structure.

Fig. 2

Linked list types ranked by traversal direction and memory usage.



The image compares linked list types based on traversal direction and memory usage. It shows singly, doubly, circular linked lists, and skip lists, explaining how each traverses nodes and how much memory they use, from simple forward traversal to layered fast searching.

3.5 Variants In Practice

Beyond the basic types, real systems often use small variants to reduce errors and improve performance. These variants keep the same core idea, but they adjust the details of how nodes connect and how edge cases are handled.

3.5.1 Sentinel Nodes

A sentinel node is a special node that does not store a real user value. It is placed at the front, the back, or both. Because it always exists, it can reduce special cases when inserting or deleting at the ends. For example, if you always have a sentinel head, then an empty list still has one node, and the first real element is after it.

3.5.2 Skip Lists Brief

A skip list is a layered form of linked list that adds additional forward links so that searching can skip over many nodes. It is not always taught in beginner courses, but it shows how the same linking idea can scale. By using multiple levels, a skip list can achieve fast average search times while staying simpler than balanced trees in some implementations.

3.6 Comparison With Arrays

Linked lists are often taught next to arrays because both can store a sequence, yet they behave differently. An array usually stores items in one continuous block of memory. This supports fast indexing: when you ask for item number i , the program can compute its address directly. In a linked list, there is no direct jump to the i -th node. The program must start from the head and follow links until it reaches the desired position. This is why random access is a strength of arrays but a weakness of linked lists.

A linked list can still be the better choice when the main work is insertion and deletion in the middle of the sequence. In an array, inserting at the front or middle requires shifting many items to create space. That shifting cost grows as the array becomes large. In a linked list, you can insert by updating a small number of links, once you have reached the correct location. The time to reach the location may still be large, but the act of insertion does not require copying a whole block of values.

3.6.1 When Lists Are A Good Fit

Linked lists fit best when a program often adds or removes items, and when it is acceptable to visit items in order. Examples include a queue of jobs where tasks arrive and leave, or a playlist where songs are inserted and removed while the user browses forward and backward. Lists are also useful when the program cannot predict the final size, and resizing an array would be expensive or awkward.

3.6.2 When Lists Are A Poor Fit

Linked lists can be a poor fit when the main goal is fast search, frequent indexing, or heavy scanning in tight loops. Each node adds extra memory overhead for links, and pointer chasing can reduce processor efficiency. Many modern programs prefer dynamic arrays, such as vectors or array lists, because they keep data together and allow the processor to use caches effectively. An academically sound conclusion is that no structure is “best” in all settings; the best choice depends on the pattern of operations and the cost model of the hardware.



4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Building A Simple List Of Tasks

Consider a small program that stores a to do list. Each task has a short description and a due date. In an array based approach, you might need to decide how many tasks you can hold. If you guess too low, you must resize the array or reject new tasks. With a linked list, you can add tasks as long as memory exists.

Start with an empty list where head is null. When the user adds the first task, create a node and set head to that node. When the user adds a second task, create a new node and connect the old last node's next field to the new node. This operation requires reaching the tail, which may take time because you walk through nodes, but the insertion does not require moving other tasks.

A simple delete operation shows the list's nature. If you want to remove a task in the middle, you find the node before it, set that node's next to skip over the removed node, and then free or discard the removed node. This changes only one link in a singly linked list. The rest of the list stays in place.

4.2 Example: Stack And Queue With Links

Stacks and queues are abstract data types that can be built with a linked list. For a stack, you want last in, first out behavior. The simplest design uses the head as the top of the stack. To push, create a new node, set its next to the current head, and update head to the new node. To pop, remove the head node and set head to the next node. Both operations are constant time because they change only a few links.

For a queue, you want first in, first out behavior. A linked list can support this efficiently if you keep both head and tail references. To enqueue, create a node and attach it at the tail, then update the tail reference. To dequeue, remove the head node and update head. Again, these operations are constant time when head and tail are maintained.

This example matters academically because it shows how a linked list can act as a flexible foundation. Instead of writing separate storage logic for each abstract type, you reuse the linking idea and adjust only which end you operate on.

4.3 Case Study: Music Playlist

A music player playlist often supports moving forward and backward through songs. A doubly linked list fits this pattern well. Each song node stores song metadata, plus a next pointer and a previous pointer. When the user presses "next," the program follows the next pointer. When the user presses "back," it follows the previous pointer.

Insertion is also common in playlists. If the user drags a new song into the middle, the player can create a node and splice it between two existing nodes by changing four links: the previous node's next, the next node's previous, and the new node's two links. No large copy is needed, so the operation stays efficient even for long playlists.

A small design choice can improve usability. If the playlist repeats, a circular doubly linked list can link the tail back to the head. Then the “next” action from the last song naturally cycles to the first. The program still needs a clear stopping rule when the user wants playback to end, but the circular structure maps cleanly to the idea of looping music.

4.4 Case Study: Memory Management In Operating Systems

Operating systems must keep track of which parts of memory are free and which are used. One classic approach is a free list, often implemented as a linked list. Each free block is represented by a node that stores the block's start address and size, and the list links free blocks together.

When a process requests memory, the system can search the list for a block that is large enough. If it finds one, it may split the block, allocate part to the process, and keep the remainder as a smaller free block. When a process releases memory, the system adds the freed block back to the list and may merge it with neighboring free blocks. These operations depend on pointer updates and careful checks.

This case study shows both strengths and limits. The list structure is flexible and can handle blocks of many sizes. However, searching for a good block can be slow if the list is long. For this reason, real systems may use multiple lists, balanced trees, or other structures for faster allocation, but linked lists remain a core idea in many designs.

4.5 Case Study: Graph Adjacency Lists

Graphs model relationships, such as roads between cities or connections between users in a social network. A common way to store a graph is an adjacency list representation. For each vertex, the program stores a list of its neighboring vertices. Each neighbor list can be built as a linked list, especially when the number of neighbors varies widely.

In this design, adding an edge can mean adding a node to one or two lists, depending on whether the graph is directed. Removing an edge means finding the matching node and splicing it out. While arrays can also store neighbors, linked lists make insertion and deletion simpler when updates happen often.

From an academic viewpoint, adjacency lists illustrate that linked lists are not only about “a list of values.” They are also about expressing relationships and building larger structures. The same node and link concept can model sequences, sets, and connections in many domains.

1. A linked list is a chain of nodes, and each node carries data plus at least one link.
2. The head reference is the starting point, and the tail is the last node in a linear list.
3. Insertion and deletion can be efficient because nodes do not need to move in memory.
4. Searching and indexing can be slower because the list is traversed one step at a time.
5. A singly linked list stores one link, while a doubly linked list stores two links.
6. A circular list connects the end back to the start and supports repeating cycles.
7. Sentinel nodes reduce special cases at the beginning and end of the list.
8. Maintaining a tail reference can make queue style operations fast.
9. Pointer safety matters: broken links can cause crashes or lost nodes.
10. Real systems often mix lists with other structures for better search performance.
11. Choosing a list type should match the operations you do most often.
12. Tracing links carefully is the best way to debug list behavior.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning	Academic Use In Context
Linked list	A chain of connected items	A linked list is a dynamic sequence defined by references.
Node	One item unit	Each node stores data and one or more links.
Link	A reference to another node	The link field defines the list's structure.
Head	The first node reference	Algorithms often start traversal from the head.
Tail	The last node	A tail reference can speed up insertion at the end.
Null	"No next item" marker	A null link signals the end of a linear list.
Pointer	Memory address holder	In C, pointers implement links between nodes.
Reference	Managed pointer concept	In Java, references connect objects safely.
Traversal	Walking through nodes	Traversal follows links to visit each element.
Insertion	Adding a node	Insertion updates links to splice in a new node.
Deletion	Removing a node	Deletion reconnects neighbors and frees memory.
Singly linked	One-direction list	A singly linked list supports forward traversal.
Doubly linked	Two-direction list	A doubly linked list supports backward traversal.
Circular	End connects to start	Circular lists model repeating processes.
Sentinel	Extra helper node	Sentinels reduce boundary condition errors.

Locality	Nearby memory benefit	Poor locality can slow linked list scans.
Big O	Growth rate notation	Big O expresses how list operations scale.

